AI Assignment 4: Travelling Salesman Problem (TSP) Using Greedy Technique and Simulated Annealing

Student ID: 2205167

September 2026

---

1 Introduction and Problem Overview

The Travelling Salesman Problem (TSP) is a classic NP-hard combinatorial optimization problem in computer science and artificial intelligence. Given a set of N cities indexed 0 through $N-1$ and a travelling cost matrix C where $C(i,j)$ represents the cost of moving from city i to city j, the goal is to discover a Hamiltonian cycle (tour) that:

1. Begins at designated starting City 0.
2. Visits every city $i \in \{0, 1, \ldots, N-1\}$ exactly once.
3. Returns to starting City 0.
4. Minimizes the total journey cost:

$$\text{Total Cost} = C(0, \pi_1) + \sum_{k=1}^{N-2} C(\pi_k, \pi_{k+1}) + C(\pi_{N-1}, 0) \tag{1}$$

where $(\pi_1, \pi_2, \ldots, \pi_{N-1})$ is a permutation of $\{1, 2, \ldots, N-1\}$.

In this assignment, we implement and evaluate two contrasting approaches:

1. **Greedy Technique (Nearest Neighbor):** A fast, deterministic constructive heuristic.
2. **Simulated Annealing (SA):** A stochastic metaheuristic optimization technique designed to escape local optima.

2 Methodology

2.1 Part A: Greedy Technique (Nearest Neighbor)

The Greedy Nearest Neighbor algorithm constructs a complete tour sequentially starting from City 0. At each step k, from current city u, it evaluates all remaining unvisited cities and moves to the unvisited city v that has the minimum travelling cost $C(u, v)$. Once all N cities are visited, the salesman returns to City 0.

Limitations: The greedy heuristic makes decisions based solely on immediate local advantage without evaluating long-term consequences. An early inexpensive move can force the algorithm into severely suboptimal, high-cost edges toward the end of the tour.

2.2 Part B: Simulated Annealing (SA)

Simulated Annealing is inspired by physical metallurgy, where heated metals are gradually cooled to form an optimal low-energy atomic crystalline structure.

- **State Representation:** A valid tour is stored as a vector $[0, \pi_1, \ldots, \pi_{N-1}]$ where City 0 remains fixed at index 0.
- **Neighborhood Operator (2-Opt / Segment Reversal):** Given two uniformly selected random positions $1 \leq i < j < N$, the segment between i and j is reversed. This operation untangles crossing paths while maintaining tour feasibility.
- **Acceptance Rule:** Let $\Delta = C_{\text{new}} - C_{\text{current}}$. If $\Delta \leq 0$ (the candidate solution is better or equal), the move is unconditionally accepted. If $\Delta > 0$ (the candidate is worse), it is accepted with Boltzmann probability:

$$P = e^{-\Delta/T} \tag{2}$$

  where T is the current temperature. A random value $r \in [0, 1]$ is drawn, accepting if $r < P$.
- **Cooling Schedule:** Geometric cooling $T_{\text{new}} = \alpha T_{\text{current}}$ is applied with cooling rate $\alpha \in (0, 1)$. At each temperature level, M iterations are executed until $T \leq T_{\min}$ or total iteration limit $I_{\max}$ is reached.

3 Experimental Evaluation and Results

We tested both algorithms on five different TSP instances generated with 2D Euclidean coordinates across varying problem sizes: $N \in \{10, 20, 50, 100, 200\}$.

3.1 Experiment 1: Greedy vs. Simulated Annealing

Default SA parameters used: $T_0 = 1000$, $\alpha = 0.995$, $T_{\min} = 0.001$, $M = 100$, $I_{\max} = 100000$, random initialization. For each instance, 5 independent SA runs were executed to obtain the best and average metrics.

Table 1: Comprehensive Comparison between Greedy Heuristic and Simulated Annealing

Instance	Cities	Greedy	SA Best	SA Avg.	Improve (%)	G Time (s)	SA Time (s)
Test 1	10	3236.70	3052.35	3052.35	5.70%	< 0.0001	0.0153
Test 2	20	4543.38	3796.30	3800.58	16.44%	< 0.0001	0.0164
Test 3	50	6917.17	5699.96	5830.16	17.60%	< 0.0001	0.0202
Test 4	100	8613.75	7956.63	8134.90	7.63%	0.0001	0.0297
Test 5	200	13243.93	12832.14	13151.19	3.11%	0.0001	0.0494

Analysis: Simulated Annealing achieved substantial cost improvements over the Greedy baseline across all test instances (up to 17.60% improvement). While Greedy runs in sub-millisecond time, SA's 2-Opt segment reversals and probabilistic uphill acceptance escape sub-optimal traps effectively.

3.2 Experiment 2: Effect of Initial Temperature (T0)

Evaluated on the 50-city benchmark instance with $\alpha = 0.995$ over 5 independent runs per parameter setting:

Table 2: Impact of Initial Temperature T_0 on SA Optimization ($N = 50$)

T_0	Best Cost	Average Cost	Accepted Moves	Worse Accepted	Avg Time (s)
100	5613.14	5664.10	1912	908	0.0201
1000	5614.90	5663.27	24816	12355	0.0194
5000	5968.09	6167.39	53984	26954	0.0200

Discussion:

- **Exploration vs. Exploitation:** A balanced initial temperature ($T_0 = 1000$) provides high early exploration followed by gradual fine-tuning.
- **High Temperature Drawback ($T_0 = 5000$):** Extremely high temperature leads to excessive acceptance of worse moves (> 26000 uphill moves), resembling a random walk and deteriorating final convergence quality.

3.3 Experiment 3: Effect of Cooling Rate (alpha)

Evaluated on the 50-city benchmark instance with $T_0 = 1000$:

Table 3: Impact of Cooling Rate α on SA Optimization ($N = 50$)

α	Best Cost	Average Cost	Total Iterations	Accepted Moves	Avg Time (s)
0.900	5922.67	6074.06	13200	1385	0.0026
0.950	5758.16	5954.74	27000	2597	0.0052
0.995	5615.23	5704.62	100000	24800	0.0201

Discussion:

- **Fast Cooling ($\alpha = 0.90$):** Quickly freezes the search into a local minimum after only 13200 iterations, resulting in worse solution quality.
- **Slow Cooling ($\alpha = 0.995$):** Allows sufficient exploration of the search landscape across temperature levels, producing the best tour cost with very fast execution time (0.020 s).

Table 4: Comparison of Random vs. Greedy Tour Initialization in SA ($N = 50$)

Initialization	Best Cost	Average Cost	Worst Cost	Avg Time (s)
Random Initial Tour	5613.14	5690.56	5855.52	0.0194
Greedy Initial Tour	5682.79	5731.86	5763.36	0.0196

3.4 Experiment 4: Random vs. Greedy Tour Initialization

Evaluated on the 50-city instance over 5 independent runs each:
Discussion: Initializing with the Greedy tour gives a much lower initial cost and narrower variance between best and worst runs. However, with slow cooling and adequate search budget, random initialization achieves slightly superior global minima by not being constrained by greedy structural biases.

4 Complexity Analysis

4.1 Greedy Technique Complexity

The Greedy Nearest Neighbor algorithm iterates $N - 1$ times. At step k, finding the closest unvisited city inspects $N - k$ candidates:

$$T_{\text{greedy}}(N) = \sum_{k=1}^{N-1} (N - k) = \frac{N(N-1)}{2} = O(N^2) \tag{3}$$

Space complexity is $O(N^2)$ to store the cost matrix and $O(N)$ for the visited array and tour vector.

4.2 Simulated Annealing Complexity

Let I be the total number of iterations executed. In each iteration:

1. **Neighbor Generation:** Reversing segment $[i, j]$ takes $O(j - i + 1) = O(N)$ time.
2. **Tour Cost Calculation:** Evaluating the complete tour takes $O(N)$ operations.
3. **Overall Time Complexity:** $O(I \times N)$ total time.

Optimization Potential: For symmetric TSP, the difference Δ can be computed in $O(1)$ arithmetic operations by evaluating only the 4 affected boundary edges:

$$\Delta = C(\pi_{i-1}, \pi_j) + C(\pi_i, \pi_{j+1}) - C(\pi_{i-1}, \pi_i) - C(\pi_j, \pi_{j+1}) \tag{4}$$

Only when a move is accepted is the tour reversed, which reduces average per-iteration overhead significantly.

5 Conclusion

- The Greedy Nearest Neighbor approach is deterministic and extremely fast ($O(N^2)$), making it well-suited for rapid tour construction.
- Simulated Annealing effectively overcomes the local minimum trap of greedy heuristics through 2-Opt segment reversals and probabilistic acceptance of worse states.
- Empirical testing demonstrates that a slow cooling rate ($\alpha = 0.995$) and moderate initial temperature ($T_0 = 1000$) consistently yield superior tour quality across all instance scales.